

محمدرضا شریفی

۴۰۱۱۰۱۹۳۲

فاز چهارم

طراحی سیستم های میکروپروسسوری

کد matrix_mult_transpose را انتخاب کرده و طبق توضیحات کلاس و اسلاید های درس، آن را cache friendly میکنیم. اسلاید درس در شکل زیر آورده شده که برای پیاده سازی کد cache friendly از آن الهام گرفته شده است.

Back to Cache: Matrix Multiplication

```
void matrix_mult_transpose (int n, double* a, double* b, double* c) {
    register double cij;
    register double *at, *bt;
    register int i, j, k;

    matrix_transpose(n, b);

    for (i=0; i<n; i++) {
        for (j = 0, bt = b; j < n; j++, c++) {
            cij = 0;

            for(k = 0, at = a; k < n; k++, at++, bt++)
                cij += *at * *bt;

            *c = cij;
        }
    }
    matrix_transpose(n, b);
}

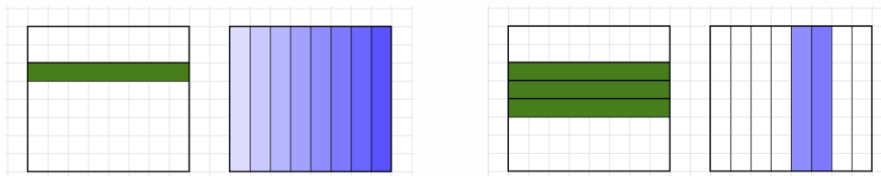
void matrix_mult_cache_friendly(int n, double* a, double* b, double* c) {
    register double cij;
    register double *at, *bt;
    register int i, j, k, jj;
    register int BS = 64;

    matrix_transpose(n, b);

    for (jj = 0; jj < n; jj += BS)
        for (i = 0; i < n; i++)
            for (j = jj; j < jj + BS && j < n; j++) {
                cij = 0.0; at = a + i * n; bt = b + j * n;

                for (k = 0; k < n; k++, at++, bt++)
                    cij += *at * *bt;

                c[i * n + j] = cij;
            }
    matrix_transpose(n, b);
}
```



کد پیاده سازی شده ی cache friendly بر اساس پیاده سازی matrix_mult_transpose :

```
// Phase 4: Cache Friendly (1D Strip Mining on Transposed Matrix)
void matrix_mult_cache_friendly(int n, double* a, double* b, double* c) {
    register double cij;
    register double *at, *bt;
    register int i, j, k, jj;
    register int BS = 64; // Block Size

    // Create a transposed copy of b to avoid mutating original
    double* b_t = (double*)_aligned_malloc(n * n * sizeof(double), 64);
    if (b_t == NULL) return;

    register double *p_b = b;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++, p_b++) b_t[j * n + i] = *p_b;
    }

    // Cache-friendly multiplication loops based on slide
    for (jj = 0; jj < n; jj += BS) {
        for (i = 0; i < n; i++) {
            for (j = jj; j < jj + BS && j < n; j++) {
                cij = 0.0;
                at = a + i * n;
                bt = b_t + j * n;

                for (k = 0; k < n; k++, at++, bt++) {
                    cij += (*at) * (*bt);
                }
                c[i * n + j] = cij;
            }
        }
    }
    _aligned_free(b_t);
}
```

نتیجه به دست آمده:

n	Matrix_mult_transpose	Matrix_multi_cache_friendly	Gain
512	0.12	0.12	1
1024	1	0.95	1.05
2048	8.75	7.51	1.16
4096	69	62.9	1.1

نتایج به دست آمده به روشنی تاثیر الگوریتم Cache-Friendly را بر سلسله مراتب حافظه نشان می دهد. در ابعاد کوچک (مانند $n=512$)، به دلیل حجم کم داده ها، پردازنده می تواند خطوط ماتریس را به راحتی در کش مدیریت کند؛ به علاوه، الگوریتم ترانهاده به لطف دسترسی خطی به حافظه و عملکرد Prefetcher های سخت افزاری به خودی خود بسیار سریع است، در نتیجه بهبود خاصی ($Gain=1$) مشاهده نمی شود. اما با بزرگتر شدن ابعاد ماتریس (محدوده ۲۰۴۸ و ۴۰۹۶)، حجم ماتریس ها به شدت از ظرفیت کش L2 و حتی کش L3 فراتر می رود. در این شرایط، الگوریتم ترانهاده ی ساده پیش از آنکه بتواند از داده های لود شده مجدداً استفاده کند، آن ها را از کش از دست می دهد. در مقابل، رویکرد Cache-Friendly با پردازش داده ها در قالب بلوک های مشخص، مجموعه داده های فعال (Working Set) را به گونه ای محدود می کند که دقیقاً در حافظه پرسرعت L1 و L2 جای بگیرند. این افزایش نرخ استفاده ی مجدد از داده ها (Data Reuse)، نیاز به واکنشی مجدد از حافظه اصلی را کاهش داده و منجر به بهبود عملکردی تا ۱۶ درصد ($Gain=1.16$) در پردازش های سنگین شده است.

N=512:

```
Do you want to run matrix_mult_cache_friendly (y/n)? y
Execution time of matrix_mult_cache_friendly = 0.12 s
```

N=1024:

```
Do you want to run matrix_mult_cache_friendly (y/n)? y
Execution time of matrix_mult_cache_friendly = 0.92 s
```

N=2048

```
Do you want to run matrix_mult_cache_friendly (y/n)? y
Execution time of matrix_mult_cache_friendly = 7.49 s
```

N=4096:

```
Do you want to run matrix_mult_cache_friendly (y/n)? y
Execution time of matrix_mult_cache_friendly = 63.15 s
```

در گام دوم، قابلیت‌های محاسبات برداری (Vectorization) را با استفاده از افزونه‌های برداری کامپایلر (GNU C Vector Extensions) در زبان C پیاده‌سازی کرده و تابع `matrix_mult_cache_friendly_vec` را توسعه می‌دهیم. از آنجایی که در قسمت قبل، نرخ خطای حافظه (Cache Miss Rate) را به حداقل رسانده و گلوگاه پهنای باند حافظه را تا حد زیادی برطرف کردیم، اکنون بستر مناسبی فراهم است تا با افزایش توان محاسباتی پردازنده، کارایی را بهبود بخشیم. با توجه به اینکه معماری سخت‌افزار مورد استفاده از دستورات AVX-256 پشتیبانی می‌کند، رجیسترهای برداری ۲۵۶ بیتی قادرند ۴ داده‌ی ممیز شناور با دقت - Double Precision 64bit را به صورت همزمان و موازی در قالب مکانیزم SIMD پردازش کنند ($256 / 64 = 4$). علاوه بر این، با تلفیق این قابلیت محاسباتی و تکنیک باز کردن حلقه (Loop Unrolling) به فاکتور ۴ (پردازش ۱۶ داده در هر تکرار)، خط لوله محاسباتی پردازنده اشباع شده و موازی‌سازی در سطح دستورات (ILP) به حداکثر می‌رسد. در نتیجه، از نظر تئوری انتظار داریم که زمان اجرای محاسبات در این بخش، گینی در حدود ۳ الی ۴ برابر را شاهد باشیم.

```

285 // Phase 4: Cache Friendly + GNU-C Vector Extensions (Max Unrolled + Aligned)
286 void matrix_mult_cache_friendly_vec(int n, double* a, double* b, double* c) {
287     register double cij;
288     register double *at, *bt;
289     register int i, j, k, jj;
290     register int BS = 64; // Block Size
291
292     // Create a transposed copy of b
293     double* b_t = (double*)_aligned_malloc(n * n * sizeof(double), 64);
294     if (b_t == NULL) return;
295
296     register double *p_b = b;
297     for (i = 0; i < n; i++) {
298         for (j = 0; j < n; j++, p_b++) b_t[j * n + i] = *p_b;
299     }
300
301     int vec_limit = n - (n % 16); // Upper limit for multiples of 16
302
303     for (jj = 0; jj < n; jj += BS) {
304         for (i = 0; i < n; i++) {
305             for (j = jj; j < jj + BS && j < n; j++) {
306                 at = a + i * n;
307                 bt = b_t + j * n;
308
309                 // Initialize 4 independent vector accumulators
310                 v4df sum0 = {0.0, 0.0, 0.0, 0.0};
311                 v4df sum1 = {0.0, 0.0, 0.0, 0.0};
312                 v4df sum2 = {0.0, 0.0, 0.0, 0.0};
313                 v4df sum3 = {0.0, 0.0, 0.0, 0.0};
314
315                 v4df *vec_at = (v4df*)at;
316                 v4df *vec_bt = (v4df*)bt;
317
318                 // Process 16 doubles at a time (4 vectors per loop)
319                 for (k = 0; k < vec_limit; k += 16) {
320                     sum0 += vec_at[0] * vec_bt[0];
321                     sum1 += vec_at[1] * vec_bt[1];
322                     sum2 += vec_at[2] * vec_bt[2];
323                     sum3 += vec_at[3] * vec_bt[3];
324
325                     vec_at += 4;
326                     vec_bt += 4;
327                 }
328
329                 // Collapse the 4 vectors into 1 vector
330                 v4df final_vec = sum0 + sum1 + sum2 + sum3;
331
332                 // Final horizontal add
333                 cij = final_vec[0] + final_vec[1] + final_vec[2] + final_vec[3];
334
335                 // --- TAIL LOOP FIX ---
336                 // Process any remaining elements if n is not a perfect multiple of 16
337                 at = (double*)vec_at;
338                 bt = (double*)vec_bt;
339                 for (; k < n; k++, at++, bt++) {
340                     cij += (*at) * (*bt);
341                 }
342
343                 c[i * n + j] = cij;
344             }
345         }
346     }
347     _aligned_free(b_t);
348 }
349

```

برای اجرای کد بالا لازم است در code blocks، باید flag را به درستی تنظیم کنیم و اطمینان حاصل کنیم کامپایلر میداند که قابلیت AVX256 در سخت افزار وجود دارد بنابراین فلگ را به شکل march=corei7-avx- تنظیم میکنیم تا از قابلیت های سخت افزار بتوان استفاده کرد .

i7 CPU (MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, Intel Core AES, PCLMUL, 64-bit extensions)

n	Matrix_mult_transpose	Matrix_mult_cache_friendly_vec	Gain
512	0.12	0.04	3
1024	1	0.28	3.57
2048	8.75	2.25	3.88
4096	69	21.10	3.27

همان طور که مشهود به گینی حدودی ۳ تا ۴ برابر در عمل رسیدیم که منطبق با انتظار نیز میباشد.

N=512:

```
Do you want to run matrix_mult_cache_friendly_vec (y/n)? y

Execution time of matrix_mult_cache_friendly_vec = 0.04 s

End Of Execution
```

N=1024:

```
Do you want to run matrix_mult_cache_friendly_vec (y/n)? y

Execution time of matrix_mult_cache_friendly_vec = 0.28 s

End Of Execution
```

N=2048:

```
Do you want to run matrix_mult_cache_friendly_vec (y/n)? y

Execution time of matrix_mult_cache_friendly_vec = 2.24 s

End Of Execution
```

N=4096:

```
Do you want to run matrix_mult_cache_friendly_vec (y/n)? y

Execution time of matrix_mult_cache_friendly_vec = 21.34 s

End Of Execution
```